

ГУАП

КАФЕДРА № 43

**ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ**

Скобцов Юрий
Александрович

должность, уч. степень,
звание

подпись, дата

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 2

Оптимизация многомерных функций с помощью генетического алгоритма

Вариант 4

по курсу: Эволюционные методы проектирования программно-информационных систем

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ

гр. №

4333К

Могилатов Семен
Игоревич

подпись, дата

инициалы,
фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 ЗАДАНИЕ И ПОСТАНОВКА ЗАДАЧИ	4
1.1 Задание	4
2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ.....	5
2.1 Общая схема генетического алгоритма	5
2.2 Операторы и остановка	5
3 РЕАЛИЗАЦИЯ И ХОД ВЫПОЛНЕНИЯ	6
3.1 Структура программы	6
3.2 Результаты базовых запусков	6
4 ИССЛЕДОВАНИЕ ПАРАМЕТРОВ.....	8
4.1 Влияние размера популяции.....	8
4.2 Влияние вероятности кроссовера.....	8
4.3 Влияние вероятности мутации	8
5 ВЫВОДЫ.....	10
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	11
Приложение А. Листинг программы	12

ВВЕДЕНИЕ

Генетические алгоритмы применяются для поиска решений в задачах, где пространство вариантов велико, целевая функция имеет сложную форму или аналитические методы неудобны. В лабораторной работе исследуется вещественный генетический алгоритм для поиска глобального минимума многомерной функции варианта 4.

Цель работы — реализовать генетический алгоритм для оптимизации перемещенного осевого гиперэллипсоида, исследовать влияние основных параметров алгоритма и сравнить полученные результаты с независимым библиотечным эволюционным оптимизатором.

В отчете приведены постановка задачи, теоретические сведения, описание операторов, листинг программы, результаты запусков для двух- и трехмерного случаев, графики сходимости и параметрического исследования, а также выводы.

1 ЗАДАНИЕ И ПОСТАНОВКА ЗАДАЧИ

1.1 Задание

Согласно методическим указаниям необходимо создать программу поиска минимума функции варианта 4, вывести график функции и точки популяции для $n=2$, исследовать влияние размера популяции и вероятностей кроссовера и мутации, а затем повторить поиск для $n=3$ и сравнить результаты.

Вариант 4 — перемещенный осевой гиперэллипсоид. В таблице варианта указаны глобальный минимум $f(x)=0$ и координаты оптимума $x_i=5i$. Поэтому в реализации используется согласованная с этими условиями функция:

$$f(x) = \sum_{i=1}^n [5i \cdot (x_i - 5i)^2].$$

Аналитический оптимум имеет вид $x^*=(5, 10, \dots, 5n)$, а минимальное значение равно $f(x^*)=0$. Для $n=2$ это точка $(5, 10)$, для $n=3$ — точка $(5, 10, 15)$. Диапазон поиска выбран $[-20; 20]$ для каждой координаты: он содержит аналитические оптимумы обоих исследуемых размерностей.

Таблица 1 — Параметры базового запуска

Параметр	Значение
Размерность n	2 и 3
Размер популяции	80
Вероятность кроссовера	0,90
Вероятность мутации	0,08
Кроссовер	BLX-alpha, alpha=0,5
Элитизм	2 особи
Максимум поколений	500
Остановка по стабильности	80 поколений
Диапазон генов	$[-20; 20]$

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Общая схема генетического алгоритма

Генетический алгоритм поддерживает популяцию альтернативных решений. На каждом шаге вычисляется значение целевой функции, более перспективные особи получают преимущество при выборе родителей, а скрещивание и мутация создают новые решения. Такая схема описана в методических указаниях и в учебном пособии Э. Вирсански (1, 2).

В отличие от двоичного представления, вещественная хромосома непосредственно содержит значения переменных. Поэтому длина хромосомы совпадает с размерностью задачи, а операции кодирования и декодирования не требуются.

2.2 Операторы и остановка

В программе используется турнирный отбор: из случайно выбранной группы кандидатов в родители попадает особь с наименьшим значением функции. Для вещественных хромосом применяется BLX-alpha кроссовер. Потомок выбирается из расширенного интервала между соответствующими генами родителей. Мутация добавляет к отдельным генам нормальное случайное возмущение, после чего координаты ограничиваются допустимым диапазоном.

Элитизм сохраняет две лучшие особи при формировании нового поколения. Критерий остановки срабатывает при достижении 500 поколений или после 80 поколений без улучшения. Фиксированные seed обеспечивают воспроизводимость серий экспериментов.

3 РЕАЛИЗАЦИЯ И ХОД ВЫПОЛНЕНИЯ

3.1 Структура программы

Файл `variant4_ga.py` содержит функцию целевой задачи, операторы отбора, BLX-alpha кроссовера и мутации, основной цикл `run_ga` и независимый запуск SciPy `differential_evolution`. Скрипт `run_experiments.py` выполняет серии повторных запусков, рассчитывает медианные показатели и создает графики.

Порядок одного поколения таков: оценить популяцию; обновить лучшую найденную особь; сохранить элиту; многократно выбрать пару родителей; с заданной вероятностью выполнить кроссовер; мутировать потомка; ограничить гены; сформировать новое поколение; увеличить счетчик стабильных поколений.

3.2 Результаты базовых запусков

В таблице 2 приведены медианные значения по 10 запускам собственного ГА и 5 запускам независимого метода SciPy. Для собственного ГА дополнительно указано расстояние до аналитического оптимума.

Таблица 2 — Сравнение результатов для $n=2$ и $n=3$

n	Метод	f	δ	ген.	t, с
2	Собственный ГА	1.2e-11	1.2e-06	107	0.243
2	SciPy differential evolution	1.0e-15	—	39	0.008
3	Собственный ГА	1.7e-11	1.3e-06	124	0.288
3	SciPy differential evolution	9.3e-14	—	298	0.051

Собственная реализация для обеих размерностей достигла значения целевой функции порядка 10^{-11} и расстояния до аналитической точки порядка 10^{-6} . При увеличении размерности медианное число поколений увеличилось с 107 до 124, а время запуска — с 0,243 до 0,288 с. Это соответствует росту числа вычисляемых генов и оценок функции.

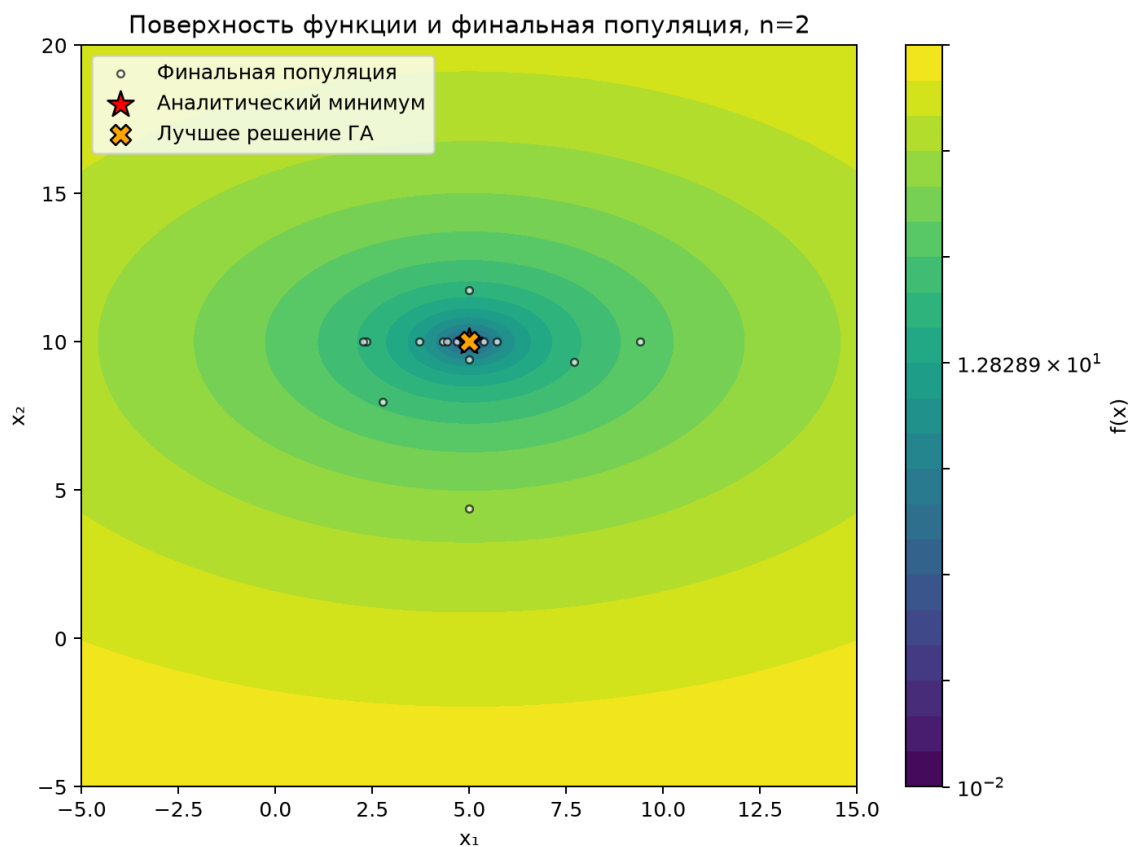


Рисунок 1 — Поверхность целевой функции и финальная популяция для $n=2$

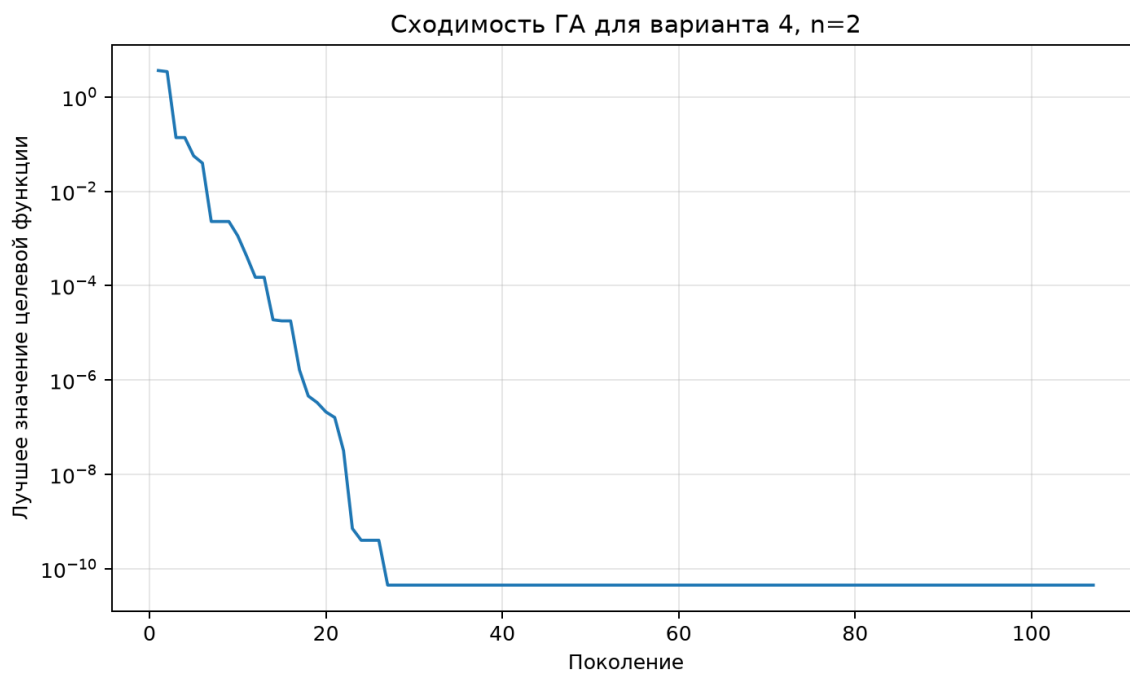


Рисунок 2 — Сходимость собственного ГА для $n=2$

4 ИССЛЕДОВАНИЕ ПАРАМЕТРОВ

4.1 Влияние размера популяции

Размер популяции исследован при значениях 30, 60 и 120 особей. При увеличении популяции время одного запуска закономерно возрастает, поскольку в каждом поколении требуется вычислить целевую функцию для большего числа особей. При этом число поколений и точность изменяются не монотонно: большая популяция дает больше разнообразия, но не всегда уменьшает число поколений на простой выпуклой функции.

4.2 Влияние вероятности кроссовера

Для вероятности кроссовера рассмотрены значения 0,60, 0,80 и 1,00. На исследуемой функции наилучший компромисс по медианной точности и числу поколений получен при 0,80. Полный кроссовер не дал преимуществ по времени, так как увеличивает интенсивность создания новых особей без существенной необходимости для выпуклой задачи.

4.3 Влияние вероятности мутации

Вероятность мутации изменялась от 0,02 до 0,20. Малое значение ускоряет локальную настройку, но уменьшает разнообразие. Значение 0,20 сохраняет возможность выхода из неудачных областей, однако увеличивает разброс и число поколений. В базовом запуске использовано промежуточное значение 0,08.

Таблица 3 — Медианные показатели параметрического исследования

Фактор	Значение	t, с	ген.	f
N	30	0.094	114.5	6.6e-11
N	60	0.182	107.5	5.6e-11
N	120	0.369	105.0	5.8e-11
p_c	0.6	0.226	111.5	3.4e-11
p_c	0.8	0.242	109.0	1.4e-11
p_c	1	0.254	107.0	4.5e-11
p_m	0.02	0.239	105.0	2.5e-11
p_m	0.08	0.245	107.5	2.7e-11
p_m	0.2	0.272	117.0	2.2e-11

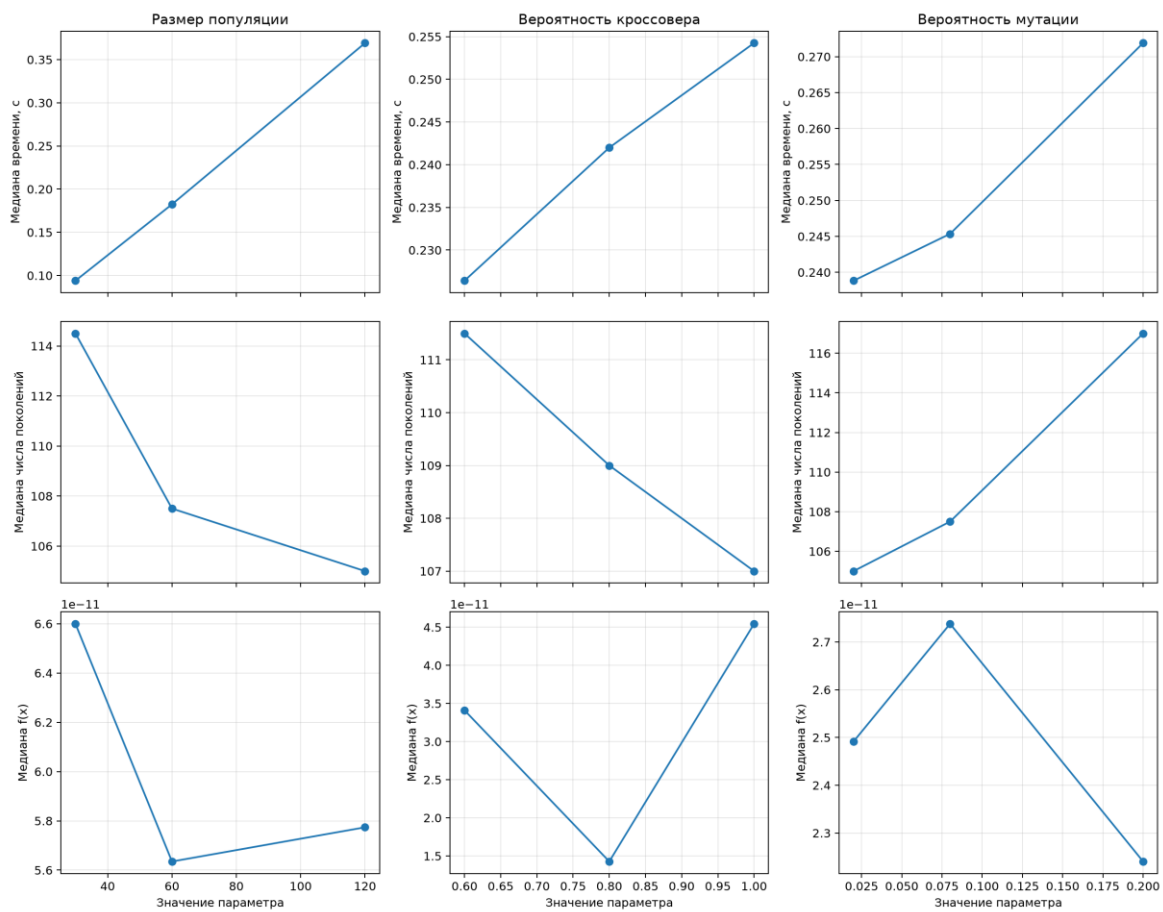


Рисунок 3 — Зависимость времени, числа поколений и точности от параметров ГА

5 ВЫВОДЫ

В ходе лабораторной работы реализован вещественный генетический алгоритм для варианта 4 — перемещенного осевого гиперэллипсоида. Использование функции $f(x) = \sum 5i(x_i - 5i)^2$ согласует указанную в методичке точку минимума $x_i = 5i$ со значением $f(x) = 0$.

Для $n=2$ и $n=3$ собственная реализация стабильно находила окрестность аналитического минимума. Медианное значение целевой функции составило $1,21 \cdot 10^{-11}$ для $n=2$ и $1,69 \cdot 10^{-11}$ для $n=3$. Сравнение с независимым эволюционным методом SciPy подтвердило корректность порядка найденных значений.

Исследование параметров показало ожидаемый рост времени при увеличении размера популяции. Вероятности кроссовера и мутации влияют на баланс между разнообразием и локальной настройкой, поэтому выбор промежуточных значений 0,90 и 0,08 обеспечивает устойчивый результат для данной выпуклой функции.

Все эксперименты сохранены в CSV и JSON, а исходный код приведен в приложении А. Благодаря фиксированному seed результаты можно воспроизвести повторным запуском `run_experiments.py`.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лабораторная работа № 2. Оптимизация многомерных функций с помощью ГА: методические указания. Приложение 1, вариант 4.
2. Вирсански Э. Генетические алгоритмы на Python / пер. с англ. А. А. Слинкина. — М.: ДМК Пресс, 2020. — 286 с.
3. ГУАП. Для учебного процесса. Нормативная документация ГУАП. — URL: <https://guap.ru/c/regdocs/docs/uch> (дата обращения: 27.09.2026).
4. SciPy documentation. `scipy.optimize.differential_evolution`. — URL: https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.differential_evolution.html (дата обращения: 27.09.2026).

Приложение А. Листинг программы

```
"""Real-coded genetic algorithm for laboratory work 2, variant 4."""

from __future__ import annotations

from dataclasses import dataclass
import time
from typing import Iterable

import numpy as np

DEFAULT_BOUNDS = (-20.0, 20.0)

def target_point(n: int) -> np.ndarray:
    """Return the analytical optimum  $x_i = 5 \cdot i$  for one-based  $i$ ."""

    if n < 1:
        raise ValueError("n must be positive")
    return 5.0 * np.arange(1, n + 1, dtype=float)

def moved_axis_parallel_hyperellipsoid(x: Iterable[float]) -> float:
    """Evaluate the shifted, weighted hyperellipsoid from variant 4.

    The methodical table names the function as shifted and declares  $x_i = 5 \cdot i$ 
    as its global minimizer. The mathematically consistent form is
     $\sum(5 \cdot i * (x_i - 5 \cdot i)^2)$ .
    """

    vector = np.asarray(x, dtype=float)
    if vector.ndim != 1 or vector.size == 0:
        raise ValueError("x must be a non-empty one-dimensional vector")
    indices = np.arange(1, vector.size + 1, dtype=float)
    return float(np.sum(5.0 * indices * (vector - 5.0 * indices) ** 2))

@dataclass(frozen=True)
class GAResult:
    best_solution: np.ndarray
    best_value: float
    generations: int
    elapsed_seconds: float
    best_history: np.ndarray
    final_population: np.ndarray

def _check_probability(value: float, name: str) -> None:
    if not 0.0 <= value <= 1.0:
        raise ValueError(f"{name} must be in [0, 1]")

def _clip(vector: np.ndarray, bounds: tuple[float, float]) -> np.ndarray:
    low, high = bounds
    return np.clip(vector, low, high)
```

```

def tournament_select(
    population: np.ndarray,
    values: np.ndarray,
    rng: np.random.Generator,
    tournament_size: int = 3,
) -> np.ndarray:
    """Select one individual for minimization by a tournament."""

    if tournament_size < 2:
        raise ValueError("tournament_size must be at least 2")
    candidates = rng.integers(0, len(population), size=tournament_size)
    winner = candidates[np.argmin(values[candidates])]
    return population[winner].copy()

def blx_alpha_crossover(
    parent_a: np.ndarray,
    parent_b: np.ndarray,
    rng: np.random.Generator,
    alpha: float = 0.5,
    bounds: tuple[float, float] = DEFAULT_BOUNDS,
) -> np.ndarray:
    """Create one child with the BLX-alpha real-valued crossover."""

    if alpha < 0.0:
        raise ValueError("alpha must be non-negative")
    lower = np.minimum(parent_a, parent_b)
    upper = np.maximum(parent_a, parent_b)
    span = upper - lower
    child = rng.uniform(lower - alpha * span, upper + alpha * span)
    return _clip(child, bounds)

def mutate(
    individual: np.ndarray,
    rng: np.random.Generator,
    mutation_probability: float,
    bounds: tuple[float, float] = DEFAULT_BOUNDS,
    scale: float = 0.10,
) -> np.ndarray:
    """Mutate genes with a bounded Gaussian perturbation."""

    _check_probability(mutation_probability, "mutation_probability")
    if scale <= 0.0:
        raise ValueError("scale must be positive")
    low, high = bounds
    result = individual.copy()
    mask = rng.random(result.size) < mutation_probability
    result[mask] += rng.normal(0.0, scale * (high - low), size=mask.sum())
    return _clip(result, bounds)

def run_ga(
    n: int,
    *,
    population_size: int = 80,
    crossover_probability: float = 0.90,
    mutation_probability: float = 0.08,
    max_generations: int = 500,
    patience: int = 80,
    tolerance: float = 1e-10,
    elite_count: int = 2,
    tournament_size: int = 3,
    alpha: float = 0.5,
    bounds: tuple[float, float] = DEFAULT_BOUNDS,
    seed: int | None = None,
) -> GAResult:

```

```

"""Run the real-coded GA and return the best individual and diagnostics."""

_check_probability(crossover_probability, "crossover_probability")
_check_probability(mutation_probability, "mutation_probability")
if population_size < max(4, elite_count + 2):
    raise ValueError("population_size is too small")
if max_generations < 1 or patience < 1:
    raise ValueError("max_generations and patience must be positive")
if elite_count < 1 or elite_count >= population_size:
    raise ValueError("elite_count must be in [1, population_size)")
low, high = bounds
if low >= high:
    raise ValueError("bounds must be ordered as (low, high)")

rng = np.random.default_rng(seed)
population = rng.uniform(low, high, size=(population_size, n))
best_solution = population[0].copy()
best_value = float("inf")
history: list[float] = []
stagnant_generations = 0
started = time.perf_counter()

for generation in range(1, max_generations + 1):
    values = np.asarray(
        [moved_axis_parallel_hyperellipsoid(individual) for individual in population]
    )
    order = np.argsort(values)
    current_value = float(values[order[0]])
    if current_value < best_value - tolerance:
        best_value = current_value
        best_solution = population[order[0]].copy()
        stagnant_generations = 0
    else:
        stagnant_generations += 1
    history.append(best_value)

    if generation == max_generations or stagnant_generations >= patience:
        break

    next_population = [population[index].copy() for index in order[:elite_count]]
    while len(next_population) < population_size:
        parent_a = tournament_select(population, values, rng, tournament_size)
        parent_b = tournament_select(population, values, rng, tournament_size)
        if rng.random() < crossover_probability:
            child = blx_alpha_crossover(parent_a, parent_b, rng, alpha, bounds)
        else:
            child = parent_a
        next_population.append(mutate(child, rng, mutation_probability, bounds))
    population = np.asarray(next_population)

return GAResult(
    best_solution=best_solution,
    best_value=best_value,
    generations=generation,
    elapsed_seconds=time.perf_counter() - started,
    best_history=np.asarray(history),
    final_population=population,
)

def run_scipy_reference(
    n: int,
    *,
    population_size: int = 80,
    max_generations: int = 500,
    seed: int | None = None,
    bounds: tuple[float, float] = DEFAULT_BOUNDS,
) -> GAResult:

```

```

"""Run SciPy's independent differential-evolution reference optimizer."""

from scipy.optimize import differential_evolution

started = time.perf_counter()
result = differential_evolution(
    moved_axis_parallel_hyperellipsoid,
    bounds=[bounds] * n,
    popsize=max(2, population_size // (15 * n)),
    maxiter=max_generations,
    polish=True,
    seed=seed,
    tol=1e-10,
    updating="immediate",
)
history = np.asarray([float(result.fun)])
return GAResult(
    best_solution=np.asarray(result.x, dtype=float),
    best_value=float(result.fun),
    generations=int(result.nit),
    elapsed_seconds=time.perf_counter() - started,
    best_history=history,
    final_population=np.asarray([result.x], dtype=float),
)

```